OmniSearch

A Cost-Based Multimodal Hybrid Search Engine

Keyword + vector retrieval over text, images, audio and video, with structured filters, a query language, a cost-based optimizer and instance-optimal top-k fusion

Srinjoy Das

Roll No. 22CS30054

Advanced Database Systems — Course Project Report

October 2026

Abstract

Keyword search misses meaning, vector search misses exact terms, and real collections mix text, images, audio and video with structured attributes. OmniSearch is a hybrid retrieval engine that treats this as a *database* problem. Every modality is turned into searchable text (OCR, captions, transcripts) and into vectors, stored in purpose-built indexes, and queried through OmniQL, a small declarative language. A cost-based optimizer chooses the access path for each clause, and an instance-optimal top-k algorithm fuses the ranked lists. All of it is implemented from scratch in Python and numpy (about 5,700 lines, 100 tests): a compressed inverted index with BM25 and WAND; HNSW and IVF vector indexes with SQ8 and product quantization; bitmap and B^+-tree attribute indexes with equi-depth histograms; and Fagin's Threshold Algorithm and NRA. Experiments against FAISS and SQLite FTS5 show three things. First, the calibrated optimizer tracks the best filtered-vector access path across a 1000× range of filter selectivity, avoiding plans that are up to 540× slower, with recall 1.00 at every point. Second, the Threshold Algorithm returns the *exact* weighted top-k while fixed-depth fusion loses up to 31% of it. Third, equi-depth histograms with most-common values keep selectivity estimates within a median q-error of 1.00, against a maximum of 802 for equi-width histograms. A retrieval-quality study on a 268-item multimodal collection shows that no single retrieval path wins. Keyword search falls to nDCG@10 0.12 on typos and 0.50 on noisy OCR text, where character-n-gram vectors reach 0.87 and 1.00. Fusion recovers most of that gap, and score-aware fusion is more robust than rank-only RRF.

Contents

1 Introduction **3**

2 Background and related work **3**

3 System overview **5**

4 Data model and query language **5**

5 Storage and index structures **6**

5.1 Inverted index 6

5.2 Vector indexes 6

5.3 Attribute indexes and statistics 6

6 Query optimization **7**

6.1 Calibrated cost model 7

6.2 Access paths for filtered vector search 7

7 Rank fusion and top-k aggregation **7**

8 Implementation **8**

9 Experimental evaluation **9**

9.1 Setup and datasets 9
9.2 ANN indexes: recall vs. latency 10
9.3 The cost-based optimizer for filtered vector search 11
9.4 Top-k fusion algorithms 12
9.5 Keyword index 13
9.6 Attribute filters and selectivity estimation 14
9.7 Vector compression 15
9.8 Scale 16
9.9 Retrieval quality on the demo collection 17
9.10 Retrieval quality on public benchmarks 19

10 Discussion **20**

11 Limitations and future work **20**

12 Conclusion **20**

A Reproducing the results **21**

B OmniQL quick reference **22**

1 Introduction

Search over a modern collection, such as a product catalogue, a lecture archive or a photo library, asks for three kinds of matching at once. A user typing *"zenith sneaker"* wants the exact brand token matched (*lexical* matching). A user typing *"sneakr"* or *"shoe for running"* wants the meaning matched (*semantic* matching). Both usually come with predicates such as *price* $<$ *100* or *category* = *sneakers* (*structured* filtering). The content is often not text at all: a brand name printed on a product photo, a phrase on a lecture slide or words spoken in a clip.

Production systems answer this with two separate engines, a full-text engine and a vector database, glued together in application code. The interesting questions are then exactly the ones a database course is about, and the glue hides them:

- **Access-path selection.** With a selective filter, should a vector query scan the few qualifying rows exactly, search an ANN graph while skipping disallowed nodes, or search unfiltered and post-filter? The answer flips with selectivity and with the *correlation* between the filter and the query.
- **Selectivity estimation.** The choice above needs an estimate of the filter's selectivity, which needs statistics.
- **Top-k aggregation.** Combining several ranked lists into one top-k is the classic middleware problem studied by Fagin et al. [5]. Fixed-depth fusion is fast but not exact.

Contributions. OmniSearch implements the full stack, so each of these questions can be answered by experiment:

1. A **data model and query language** (OmniQL) that mixes scoring clauses (`MATCH`, `SIMILAR`) with SQL-style filters under three-valued logic (Section 4).
2. **Index structures built from scratch**: a gap + variable-byte compressed inverted index with BM25 and WAND; HNSW and IVF vector indexes with SQ8 and PQ compression and multi-vector MaxSim for video; and bitmap, B^+-tree and histogram attribute indexes (Section 5).
3. A **cost-based optimizer** whose unit costs are calibrated on the host machine. It chooses filtered-vector plans, B^+-tree vs. column scan, BM25 vs. WAND, and the top-k algorithm, and explains its choices with `EXPLAIN ANALYZE` (Section 6).
4. **Exact top-k fusion** with the Threshold Algorithm and NRA, alongside reciprocal-rank fusion (Section 7).
5. An **experimental evaluation** against FAISS and SQLite FTS5, plus a retrieval-quality study that separates the lexical, semantic and OCR paths (Section 9).

2 Background and related work

Lexical retrieval. BM25 [1] scores a document d for query terms q_i as

$$\mathrm{BM25}(d,q) = \sum_i \mathrm{IDF}(q_i)\,\frac{f(q_i,d)\,(k_1+1)}{f(q_i,d)+k_1\left(1-b+b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \qquad \mathrm{IDF}(q_i) = \ln\!\left(1+\frac{N-n_i+0.5}{n_i+0.5}\right), \tag{1}$$

with $k_1 = 1.2$ and $b = 0.75$ in OmniSearch. WAND [2] skips documents whose score upper bound cannot reach the current top-k threshold.

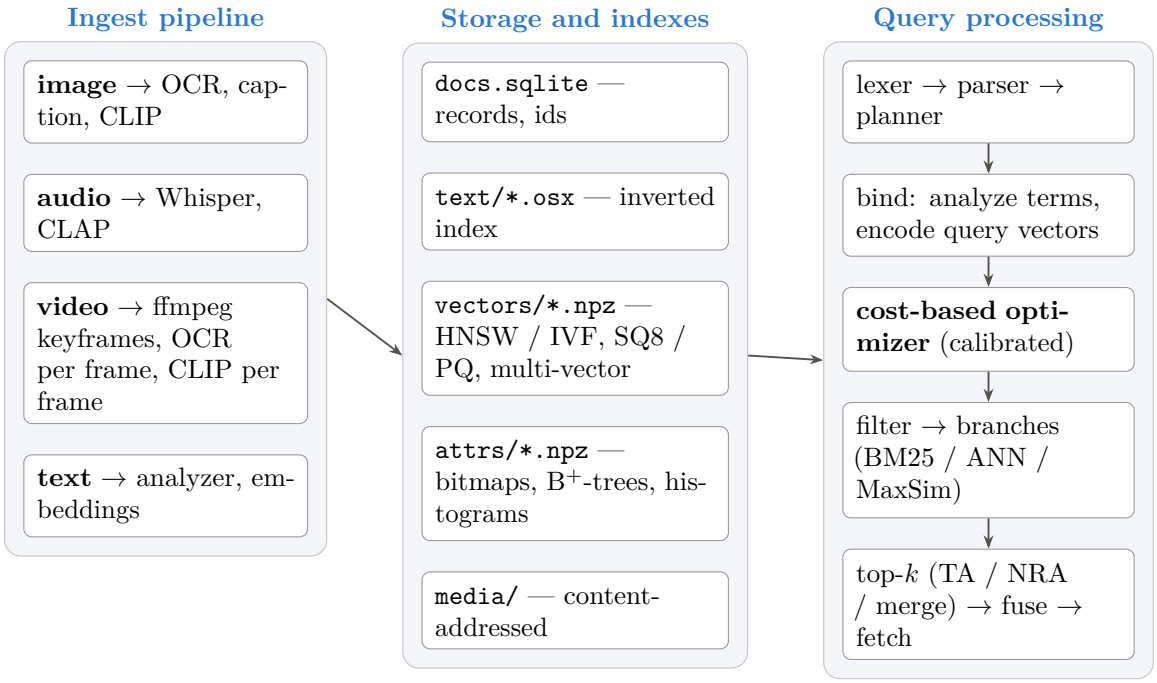


Figure 1: Architecture. Every modality is reduced to text (searched with BM25) and to vectors (searched with ANN indexes); attributes go to bitmap / B^+-tree indexes with statistics. A query is parsed, bound, optimized with calibrated costs, and executed as filter → scoring branches → top-k fusion.

Approximate nearest neighbours. HNSW [3] builds a multi-layer proximity graph and answers queries with a greedy beam search of width ef. IVF partitions vectors with k-means and scans the *nprobe* closest partitions. Product quantization [4] compresses vectors into sub-space codebook indices and computes distances from lookup tables.

Filtered vector search. Milvus [7] and AnalyticDB-V [8] combine attribute filtering with ANN search and pick between pre- and post-filtering by cost. ACORN [9] changes the HNSW construction itself so in-graph filtering stays efficient for arbitrary predicates. OmniSearch takes the optimizer route and measures where each plan breaks.

Rank aggregation. Reciprocal Rank Fusion (RRF) [6] merges lists by rank only: $\mathrm{RRF}(d) = \sum_i w_i/(k + r_i(d))$ with $k = 60$. For score-based (weighted) fusion, Fagin's Threshold Algorithm (TA) and No-Random-Access algorithm (NRA) [5] are instance-optimal for monotone aggregation functions.

Access-path selection. The optimizer follows the System R tradition [10]: an explicit cost model with unit costs (cf. PostgreSQL's `seq_page_cost` and `cpu_tuple_cost`), statistics-driven selectivity estimates, and a choice of the cheapest alternative per operator.

Multimodal encoders. With the optional neural profile, CLIP [11] maps images, video keyframes and text into one embedding space, BLIP produces captions, Whisper transcribes speech and RapidOCR (PP-OCR) reads text in images and video frames.

3 System overview

Figure 1 shows the three layers. The **ingest pipeline** routes each media field to extractors and encoders, batching model calls. Precomputed values (for example captions and vectors computed separately on a GPU) are reused, so expensive models never have to run on the indexing machine. The **storage layer** keeps records in SQLite, which maps dense internal document ids to external ids, and keeps every index in its own file format. The **query layer** turns OmniQL into a physical plan, executes it and can report estimated versus actual cost for every operator.

Two encoder profiles are provided. The *classic* profile is fully offline: hashed character n-gram text vectors, colour-layout image vectors and OCR. It needs no model downloads and is used for every experiment in this report. The *CLIP* profile adds CLIP image and keyframe embeddings, BLIP captions and Whisper transcripts, so a text query can match pixels directly.

4 Data model and query language

Schema. A collection's schema declares typed fields:

- **media** fields (image, audio, video) with a list of extractors (`ocr`, `caption`, `transcript`);
- **text** fields, the concatenation of source fields such as `title`, `description` and `image.ocr`;
- **vector** fields with an encoder, a source and an index type (`hnsw`, `ivf`, `flat`);
- **multi-vector** fields with one vector per video keyframe;
- **attribute** fields: `int`, `float`, `category` and multi-valued `tags`.

OmniQL. Queries have the form

```
[EXPLAIN [ANALYZE]] SEARCH <collection>
  [WHERE <condition>]
  [FUSE RRF(k=60) | WEIGHTED(norm='max'|'minmax'|'none') | AUTO]
  [LIMIT n]  [WITH (option=value, ...)]
```

Scoring clauses rank documents: `MATCH(field, 'text')` scores with BM25, and `SIMILAR(field, q)` scores by vector similarity. In `SIMILAR`, q may be text, `IMAGE('path')`, `AUDIO('path')`, `DOC('id')`, a literal `VECTOR[...]` or a weighted sum such as `0.7*IMAGE('a.jpg') + 0.3*'blue'`. *Filters* restrict documents: comparisons, `IN`, `BETWEEN`, `IS [NOT] NULL`, and `CONTAINS` (all terms present, evaluated by posting-list intersection), combined with `AND`/`OR`/`NOT`. Scoring clauses must be top-level conjuncts. A document qualifies if it passes every filter and appears in at least one scoring clause's results. Filters follow SQL's three-valued logic, so `NOT (price < 50)` does not match a document whose price is `NULL`. The `WITH` options override the optimizer for experiments, for example `strategy=postfilter` or `topk=nra`.

Example. The plan below was produced by the engine on the demo collection. The optimizer rejects both HNSW plans for the `SIMILAR` branch because the filter leaves only 28 candidates, and it evaluates `price <= 100` by column scan because the estimated selectivity is 75%:

```
EXPLAIN ANALYZE SEARCH demo
WHERE MATCH(text, 'zenith sneaker') AND SIMILAR(text_vec, 'zenith sneaker')
  AND category IN ('sneakers') AND price <= 100
FUSE RRF(k=60) LIMIT 12
```

```
-> Limit 12
  -> Fuse RRF(k=60)  algorithm: materialise depth 100 + merge  est 0.207 ms  [actual: 0.47 ms, candidates
     =28]
     -> [text] BM25 Scan (term-at-a-time)  k=100  terms: zenith(df=38), sneaker(df=48)  est 0.142 ms
          rejected: wand 0.417 ms (~86 pivot steps (heuristic))
     -> [text_vec] Pre-filter + Exact Vector Scan  k=100  est 0.034 ms  [actual: 0.15 ms, 28 rows]
          rejected: index 1.633 ms (~268 node expansions); postfilter 1.633 ms (fetch 268, keep ~36)
  -> Filter: category IN ('sneakers') AND price <= 100  est sel 13.53%  [actual: 28 rows, sel 10.45%]
       category IN ('sneakers'): bitmap
       price <= 100: column scan  (scan 0.000 ms vs index 0.094 ms, est sel 75.52%)
```

5 Storage and index structures

5.1 Inverted index

Text is analyzed (lower-casing, tokenization, stop-word removal, light stemming) into a term → posting-list map (`index/text/`). Document ids in a posting list are stored as *gaps*, and gaps and term frequencies are encoded with *variable-byte* coding. Decoding is vectorised in numpy (46 million integers/s), avoiding a per-byte Python loop. Indexes use a custom binary file format (`OSX1`). Two scoring algorithms are provided: term-at-a-time exhaustive BM25 (vectorised accumulation over each posting list) and WAND [2], which keeps per-term score upper bounds and skips documents that cannot enter the top-k. Random-access scoring of a single (term, document) pair supports the Threshold Algorithm.

5.2 Vector indexes

- **Exact scan**: a contiguous float32 matrix–vector product, or a product over a *gathered* subset of rows when a filter is pushed down.
- **HNSW** [3], from scratch: $M = 16$, $ef_{\text{construction}} = 100$, heuristic neighbour selection, and *in-graph filtering*. Disallowed nodes are traversed but never returned, so the beam has to expand roughly $1/s$ times more nodes at selectivity s.
- **IVF** with its own k-means++ training; a filtered search widens *nprobe* until about $4k$ allowed candidates are expected.
- **Compression**: scalar int8 quantization (SQ8, 4× smaller) and product quantization (PQm, m bytes per vector, ADC lookup tables), each optionally followed by exact re-ranking of the top $r \cdot k$ candidates.
- **Multi-vector** fields for video store one vector per keyframe and score a document by MaxSim, $\max_j \langle q, v_j \rangle$. The best keyframe's timestamp becomes the result's "best moment", which the UI uses to start playback there.

5.3 Attribute indexes and statistics

Categorical and tag attributes use **bitmap indexes**, so an `IN` list is a bitwise OR. Numeric attributes are stored as dense columns, which a vectorised scan reads, plus a **B$^+$-tree** of order 64 with bulk loading and linked leaves for range scans. For selectivity estimation, each numeric column keeps **equi-depth histograms** (64 buckets) plus the 16 **most common values** and the null fraction, refreshed automatically after enough updates. Predicates combine under the attribute-value independence assumption.

6 Query optimization

6.1 Calibrated cost model

Costs are measured in seconds and composed from **unit costs**, such as one element of a contiguous scan, one element of a gathered scan, one HNSW node expansion, one posting, one WAND pivot step, one B$^+$-tree row, one column-scan row, one sorted access and one random access. `Collection.calibrate()` times micro-operations on the host machine and fits these constants, including the coefficients a, b of the HNSW expansion model

$$E(ef) = a \cdot ef + b \cdot \log_2 N. \tag{2}$$

The constants are stored in `cost.json`. This plays the same role as PostgreSQL's planner cost constants, but the values are measured rather than hand-set. Calibration matters in an interpreted implementation, where the ratio between vectorised and interpreted operations varies by more than an order of magnitude across machines.

6.2 Access paths for filtered vector search

For a `SIMILAR` clause over N vectors of dimension d with a filter of estimated selectivity s, the optimizer compares four plans:

Plan	Estimated cost
Pre-filter + exact scan	$sN \cdot d \cdot c_{\text{gather}}$
HNSW, filter pushed into the graph	$E(ef)/s \cdot c_{\text{expand}}$
HNSW + post-filter	$E(\max(ef,\, 1.5k/s)) \cdot c_{\text{expand}}$
IVF with masked candidates	$n_{\text{list}}\, d\, c_{\text{scan}} + \frac{nprobe'}{n_{\text{list}}}\, Ns\, d\, c_{\text{gather}}$

Post-filtering over-fetches so that about $1.5k$ results survive the filter; IVF widens $nprobe'$ so that about $4k$ allowed candidates are expected. Selectivity comes from bitmap counts for categories and from histograms plus MCVs for numeric ranges. The same model also chooses

- B$^+$-tree vs. column scan for each range predicate,
- exhaustive BM25 vs. WAND for each `MATCH` clause, and
- materialise-and-merge vs. TA vs. NRA for score-based fusion.

7 Rank fusion and top-k aggregation

RRF merges branch rankings by $\sum_i w_i/(60 + r_i)$. It needs no score calibration, but it only sees each branch's top-*depth* list (default 100). **Weighted** fusion sums normalised scores, $\sum_i w_i \cdot \max(0, s_i/s_i^{\max})$ (or min–max normalisation). **AUTO** picks the weights per query from properties of the bound query, for example a query containing a model number or quoted text leans lexical.

Exact weighted top-k. Max-normalised weighted fusion is monotone, so Fagin's algorithms [5] apply. Every branch exposes *sorted access* (its next-best document) and *random access* (the score of any document):

- **TA** reads all branches in parallel by sorted access. For each newly seen document it random-accesses the missing scores. It stops once the current k-th best aggregate is at least the threshold $\tau = \sum_i w_i\, \bar{s}_i$, where $\bar{s}_i$ is the last score seen in branch i.

- **NRA** uses sorted access only. It maintains lower and upper bounds per document and stops when k documents have lower bounds that no other document's upper bound can exceed.

Both return the exact top-k; materialise-and-merge at a fixed depth is approximate whenever a true top-k document lies below that depth in some branch.

8 Implementation

Component	Where
Record and media store	`omnisearch/storage/` (SQLite, content-addressed media)
Inverted index, BM25, WAND	`omnisearch/index/text/`
HNSW, IVF, SQ8/PQ, multi-vector	`omnisearch/index/vector/`
Bitmaps, B^+-tree, histograms	`omnisearch/index/attribute/`
Query language, optimizer, EXPLAIN, top-k and fusion	`omnisearch/query/`
Encoders and ingest	`omnisearch/encoders/`, `omnisearch/ingest/`
HTTP API and web UI	`omnisearch/api/` (FastAPI, static HTML/JS)
Benchmarks and baselines	`benchmarks/`, `baselines/pgvector/`

Table 1: Code map. The engine is about 5,700 lines of Python; the test suite has 100 tests.

The engine is pure Python with numpy; there are no compiled extensions. Hot loops are vectorised: posting decode, BM25 accumulation, exact scans, column scans and PQ lookup tables. HNSW search, WAND and NRA are inherently sequential and run in the interpreter, which the experiments make visible. A FastAPI server exposes collections, search, OmniQL, document CRUD and media streaming. A web UI (Figure 2) offers text and image queries, facet filters that apply immediately, per-branch score bars, a "why it ranked" detail view, video best moments, a visual query-plan viewer and an OmniQL console.

Portability. The project was developed on Linux and ported to Windows as part of this work. The changes were:

- an OS-aware `Makefile` that uses `cmd.exe`, `.venv\Scripts\python.exe` and a Python-based `clean`;
- explicit UTF-8 encoding for every text file read and write;
- forward-slash relative media paths in dataset records;
- a cross-platform font list in the demo-data generator, which previously assumed a Linux font path.

All 100 tests pass on both platforms.

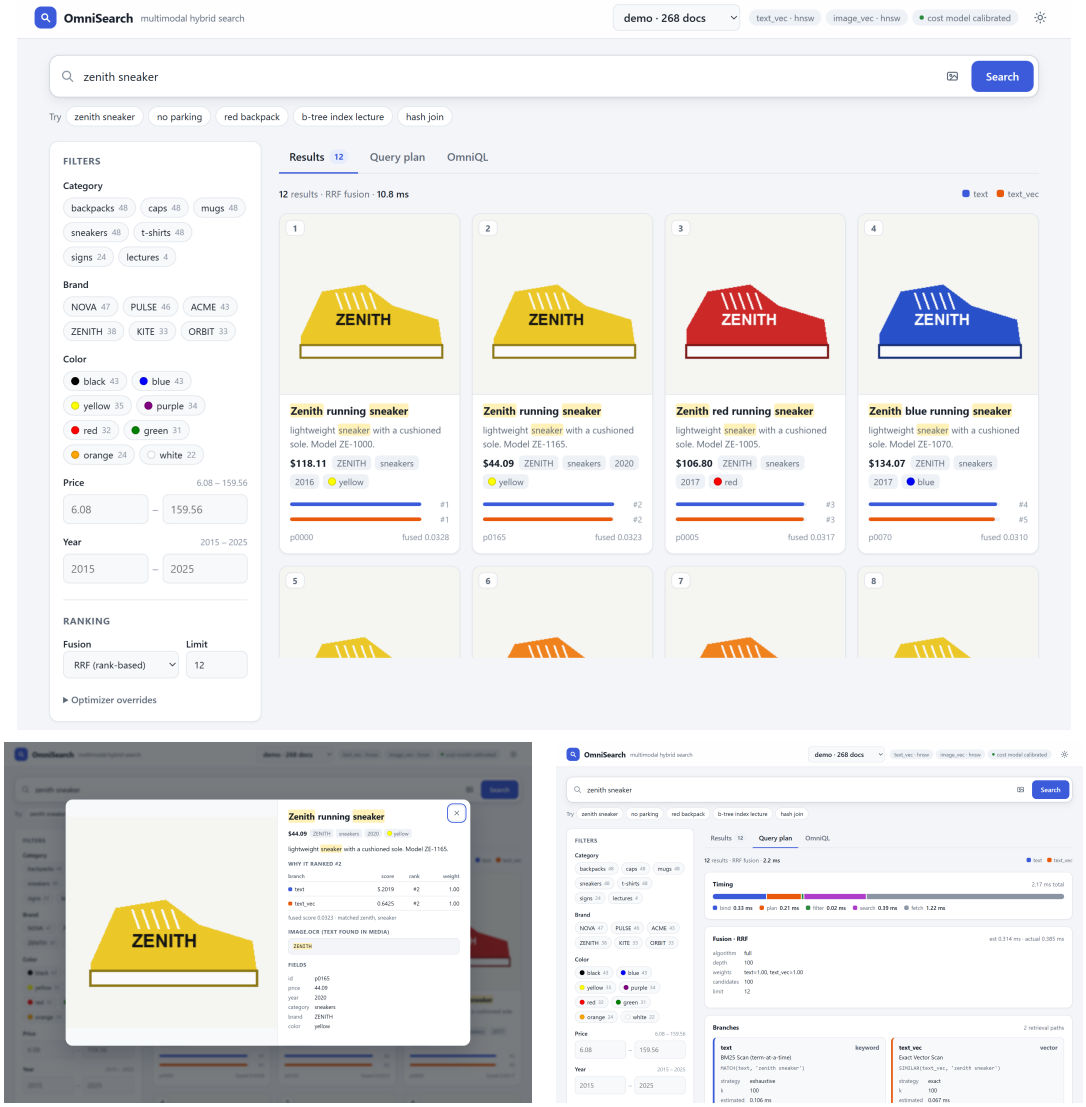


Figure 2: Web UI. Top: hybrid results for *"zenith sneaker"*, with matched terms highlighted and per-branch score bars (keyword in blue, vector in orange). Bottom left: the detail view explaining a result's rank and showing the text OCR found in the image. Bottom right: the query-plan viewer with the timing breakdown, the fusion operator and each branch's estimated vs. actual cost and rejected alternatives.

9 Experimental evaluation

9.1 Setup and datasets

Systems experiments use synthetic data with *controlled* structure. Isolating an index or optimizer effect requires knowing the true selectivity, the true neighbours and the correlation between filter and query, and real embeddings hide all three. Retrieval quality is measured on the demo collection, whose ground truth is known by construction (Table 2). Every benchmark writes JSON to `results/`, and every table in this section is generated directly from those files, so no number is copied by hand.

Dataset	Construction	Used in
Clustered vectors	$N = 20{,}000$ (50,000 for compression), $d = 128$, 64 Gaussian clusters on the unit sphere, cosine; 200 queries drawn near cluster centres; exact ground truth by brute force	§9.2, §9.3, §9.7
Topic corpus	50,000 documents, 30k-word Zipfian vocabulary; 35% of each document's words from its cluster's topic vocabulary, so lexical and vector scores are correlated; 250 queries	§9.5
Top-k corpus	20,000 documents, topic text + $d = 64$ clustered vectors; 2–3 ranked lists that are correlated (same topic) or independent	§9.4
Attribute column	200,000 rows; uniform `bucket` (exact selectivities), cluster-correlated `group`, log-normal (skewed) `price`	§9.6
Scale	10k, 25k and 50k documents with text, a 128-d vector and attributes	§9.8
Demo collection	268 items: 240 rendered product photos (6 brands × 5 categories × 8 colours; brand printed on the product; a third of titles omit the colour), 24 street-sign photos (8 texts), 4 lecture videos (4 slides each)	§9.9

Table 2: Datasets. All are generated reproducibly from fixed seeds.

Experiments	Environment
Systems benchmarks (ANN, optimizer, top-k, text, filters, compression, scale)	Python 3.13.17, numpy 2.5.4, 4 CPUs (x86_64), 2026-10-23 16:31 UTC
Demo retrieval quality	Python 3.11.9, numpy 2.4.6, 12 CPUs (AMD64), 2026-10-23 21:24 UTC

Table 3: Environments. Queries are single-threaded. FAISS 1.8 and SQLite FTS5 are the baselines.

9.2 ANN indexes: recall vs. latency

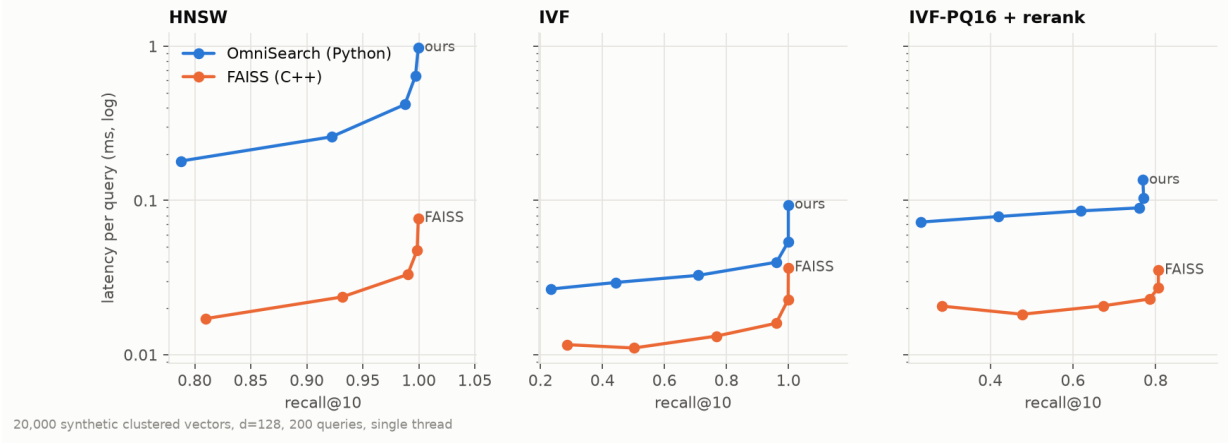


Figure 3: Recall@10 against query latency on 20,000 clustered 128-d vectors, sweeping ef (HNSW) and *nprobe* (IVF).

Index	Build (s)	Best recall@10	ms at recall ≥ 0.95
HNSW (ours)	22.30	1.000	0.419
IVF (ours)	2.51	1.000	0.040
IVF-PQ16 (ours)	7.46	0.771	–
Exact scan (ours)	0.01	1.000	0.147
FAISS HNSW	1.38	1.000	0.033
FAISS IVF	0.82	1.000	0.016
FAISS IVF-PQ	1.69	0.807	–

Table 4: Build time, best recall and the fastest setting that reaches recall ≥ 0.95.

Our HNSW reaches recall 1.00 but costs 0.42 ms per query against 0.033 ms for FAISS's C++ implementation (≈13×). Our IVF is within 2.5× of FAISS, because its inner loop is a vectorised partition scan rather than interpreted graph traversal. At 20k vectors a vectorised exact scan (0.15 ms) still beats our HNSW. The calibrated cost model knows this and chooses the exact scan for unfiltered queries at this size (Table 5, last row, is the crossover). IVF-PQ16 plateaus at recall 0.77; product quantization is weak on these isotropic synthetic vectors (Section 9.7).

9.3 The cost-based optimizer for filtered vector search

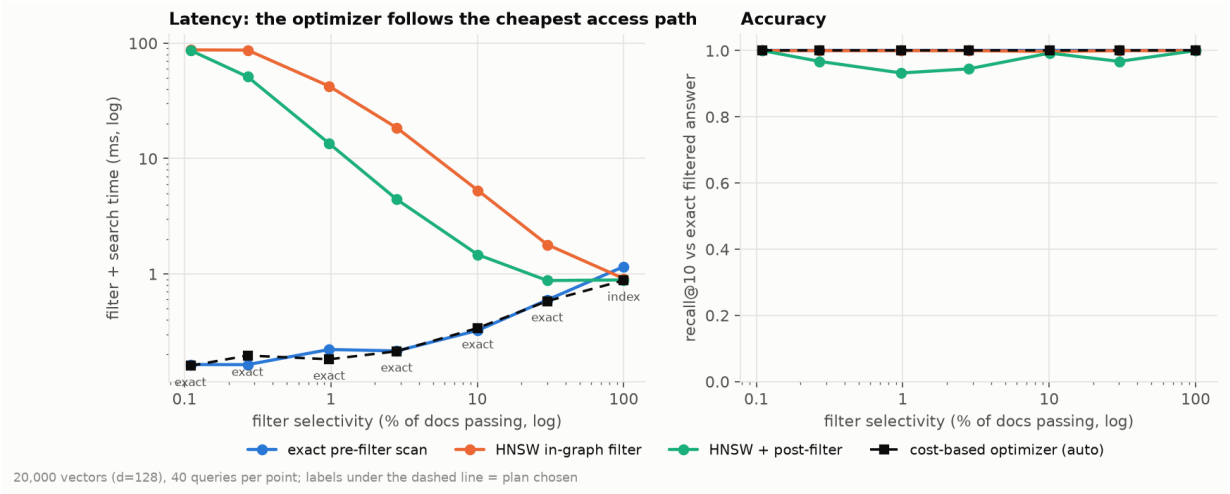


Figure 4: Latency (left) and recall (right) of each filtered access path as filter selectivity varies, and the plan chosen by the optimizer (dashed).

Selectivity	Exact	In-graph	Post-filter (rec.)	Optimizer (rec.)	Chosen
0.11%	0.16	87.53	86.71 (1.00)	**0.16** (1.00)	exact
0.27%	0.16	87.16	51.21 (0.97)	**0.20** (1.00)	exact
0.97%	0.22	42.37	13.48 (0.93)	**0.18** (1.00)	exact
2.82%	0.22	18.47	4.42 (0.95)	**0.21** (1.00)	exact
10.08%	0.32	5.30	1.47 (0.99)	**0.34** (1.00)	exact
30.09%	0.60	1.79	0.88 (0.97)	**0.58** (1.00)	exact
100.00%	1.16	0.91	0.89 (1.00)	**0.88** (1.00)	index

Table 5: Mean latency (ms) and recall@10 per access path. "Optimizer" is the plan chosen by the cost model.

Filter	Exact	In-graph	Post-filter	Optimizer
correlated	0.16 (1.00)	1.26 (1.00)	8.21 (1.00)	0.15 (1.00)
anti-correlated	0.16 (1.00)	29.12 (1.00)	8.82 (0.82)	0.20 (1.00)

Table 6: Same selectivity, different correlation: the filter selects documents in the query's own cluster (correlated) or away from it (anti-correlated). Latency in ms (recall@10).

The optimizer tracks the lower envelope. With a 0.1% filter the pre-filtered exact scan takes 0.16 ms, while HNSW with in-graph filtering takes 87.5 ms (540× slower) because the beam must expand on the order of $1/s$ nodes to find ef allowed ones. Post-filtering is just as slow, because it must over-fetch $1.5k/s$ results, and it also loses recall (0.93 at 1%). The optimizer chooses the exact scan up to 30% selectivity and switches to HNSW only for the unfiltered query, with recall 1.00 at every point (Figure 4).

Correlation matters as much as selectivity. At equal selectivity, in-graph HNSW filtering costs 1.3 ms when the allowed documents lie in the query's neighbourhood and 29 ms when they lie away from it; post-filter recall drops from 1.00 to 0.82 (Table 6). An optimizer that assumes independence cannot see this. It is the failure mode that motivates ACORN [9], and here the cost model avoids it by preferring the exact plan whenever the filtered set is small.

9.4 Top-k fusion algorithms

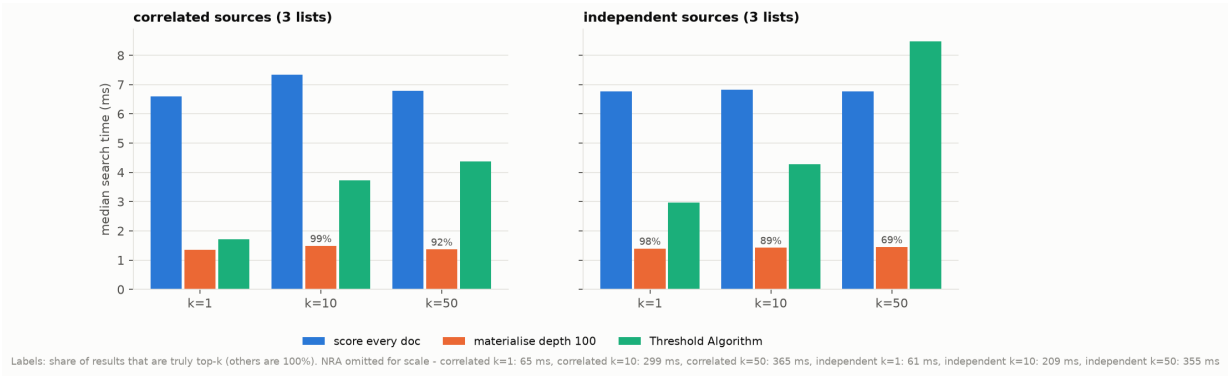


Figure 5: Weighted fusion of m ranked lists: latency, accesses and exactness of each top-k algorithm.

Lists	m	k	Exhaustive	Merge@100	TA	TA accesses	NRA
correlated	2	1	3.89	0.73 (100%)	0.32 (100%)	19+18	1.09 (100%)
correlated	2	10	3.81	0.72 (100%)	0.60 (100%)	59+57	2.91 (100%)
correlated	2	50	3.86	0.74 (100%)	1.23 (100%)	190+170	6.65 (100%)
correlated	3	1	6.60	1.36 (100%)	1.71 (100%)	159+302	65.23 (100%)
correlated	3	10	7.34	1.49 (99%)	3.73 (100%)	405+733	299.33 (100%)
correlated	3	50	6.79	1.37 (92%)	4.37 (100%)	609+1098	365.24 (100%)
independent	2	1	4.16	0.76 (100%)	0.90 (100%)	131+131	28.03 (100%)
independent	2	10	3.90	0.78 (100%)	1.71 (100%)	245+245	45.57 (100%)
independent	2	50	3.94	0.78 (99%)	1.64 (100%)	274+274	51.23 (100%)
independent	3	1	6.77	1.40 (98%)	2.97 (100%)	367+729	61.02 (100%)
independent	3	10	6.82	1.42 (89%)	4.29 (100%)	505+1002	209.41 (100%)
independent	3	50	6.77	1.45 (69%)	8.47 (100%)	951+1876	355.09 (100%)

Table 7: Latency in ms; in parentheses, the fraction of the true top-k returned. "Merge@100" materialises each list to depth 100 and merges. "TA accesses" counts sorted + random accesses.

TA is exact and stops early. With two correlated lists, TA returns the exact top-1 after only 19 sorted and 18 random accesses, in 0.32 ms against 3.89 ms for scoring every document (12×). Fixed-depth merge is fast, but it is *wrong* exactly where fusion matters most: with three independent lists it returns only 69% of the true top-50, because true top-50 documents rank below 100 in some list. TA stays at 100% in every setting. **NRA** is exact too, but it is the slowest here. It avoids random access, the right trade-off when random access is expensive (a remote service), whereas here a random access is a cheap in-memory lookup. Its per-candidate bound bookkeeping also runs in the interpreter.

9.5 Keyword index

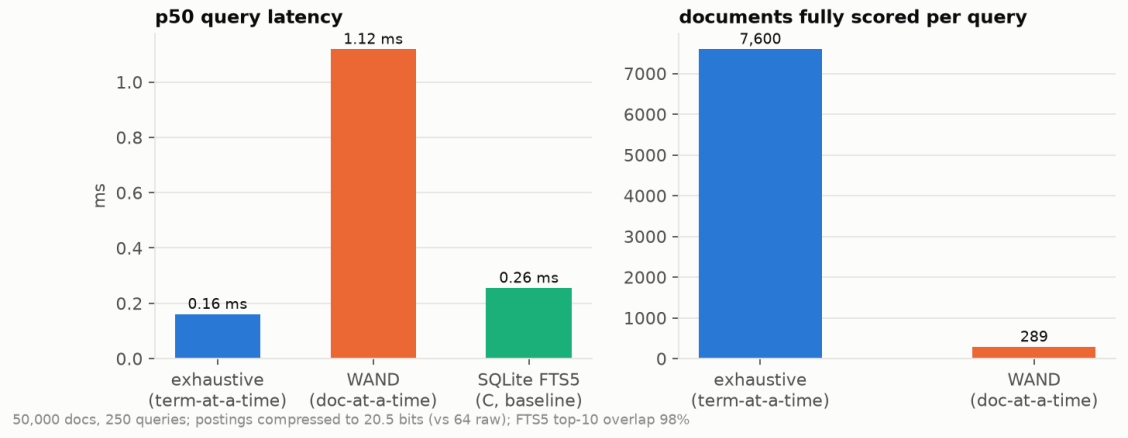


Figure 6: Posting-list compression and BM25 query latency (exhaustive vs. WAND vs. SQLite FTS5).

Metric	Value
Documents / distinct terms	50,000 / 29,083
Postings	1,057,314
Index size, compressed vs raw	2.71 MB vs 8.46 MB
Bits per posting	20.5
Vectorised VByte decode	46 M integers/s
BM25 p50 latency: exhaustive / WAND	0.16 ms / 1.12 ms
Documents fully scored: exhaustive / WAND	7,600 / 289
SQLite FTS5 p50 latency	0.26 ms
Top-10 overlap with FTS5	98%

Table 8: Inverted index on the 50k-document topic corpus.

Gap + VByte coding stores a posting in 20.5 bits, 3.1× smaller than raw 32-bit pairs. Our BM25 top-10 agrees 98% with SQLite FTS5's `bm25()` ranking, and the median latency is lower (0.16 vs. 0.26 ms). WAND scores 26× fewer documents (289 vs. 7,600), yet it is *slower* in Python (1.12 ms). Its savings are in work, but each pivot step is interpreted, while exhaustive scoring is one vectorised pass per term. The cost model includes a per-step WAND cost, so it chooses WAND only for queries with very long posting lists.

9.6 Attribute filters and selectivity estimation

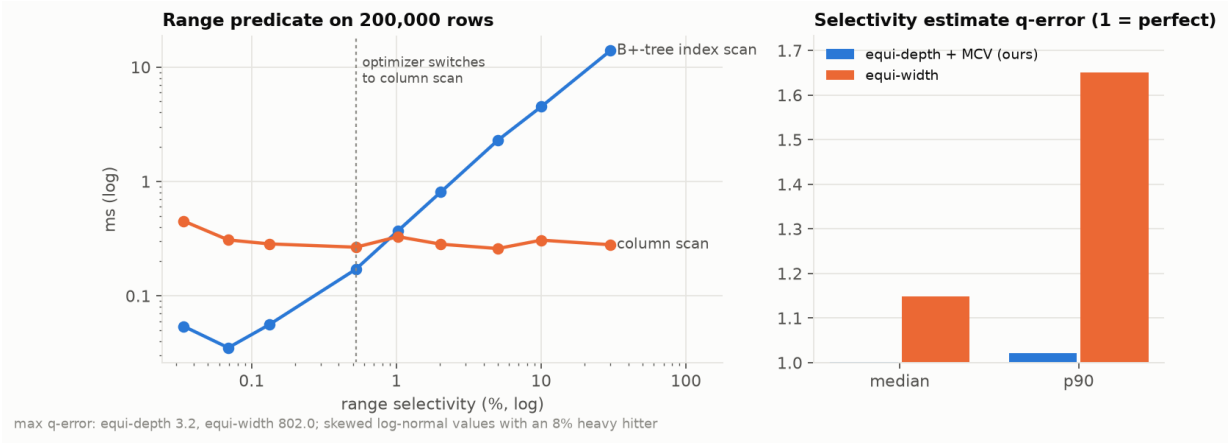


Figure 7: Left: B$^+$-tree range scan vs. vectorised column scan on 200k rows. Right: selectivity estimation error of equi-depth + MCV vs. equi-width histograms on a skewed column.

Selectivity	B+-tree (ms)	Column scan (ms)	Optimizer choice
0.034%	0.054	0.450	B+-tree
0.069%	0.035	0.308	B+-tree
0.132%	0.056	0.284	B+-tree
0.524%	0.170	0.266	column scan
1.025%	0.370	0.329	column scan
2.023%	0.806	0.283	column scan
5.020%	2.291	0.259	column scan
10.018%	4.504	0.306	column scan
30.020%	13.895	0.279	column scan

Statistics	Median	p90	Max
Equi-depth + MCV (ours)	1.00	1.02	3.2
Equi-width	1.15	1.65	802.0

Table 9: Top: B$^+$-tree vs. column scan latency per range selectivity and the optimizer's choice. Bottom: q-error of range-selectivity estimates, $\max(\hat{s}/s,\ s/\hat{s})$, on the skewed column.

The B$^+$-tree wins below about 0.5% selectivity. Above that, gathering rows one by one from the tree loses to a vectorised scan of the whole column, which costs a near-constant 0.3 ms. The optimizer switches at exactly that point. Equi-depth histograms with most-common values estimate range selectivity with a median q-error of 1.00 and a maximum of 3.2; equi-width histograms reach 802 on the skewed price column. A wrong estimate of that size would send the vector optimizer to the wrong plan in Table 5.

9.7 Vector compression

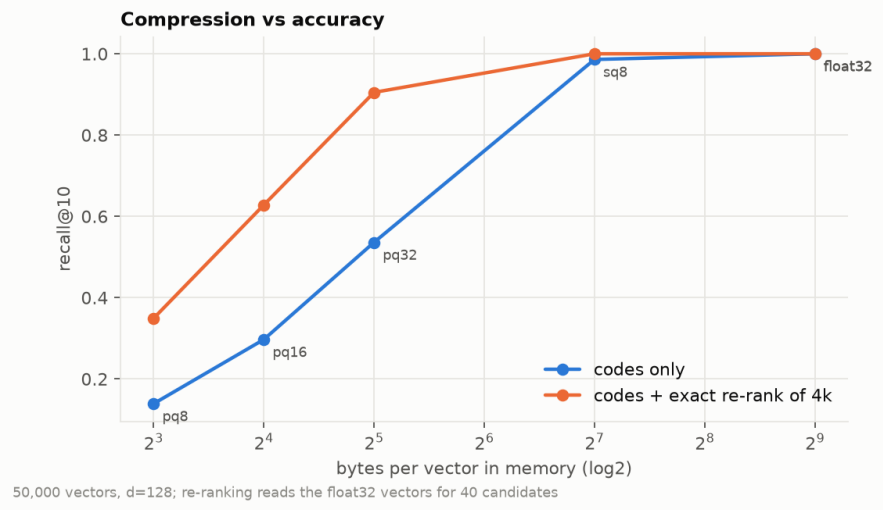


Figure 8: Memory per vector against recall@10 (50,000 vectors, $d = 128$), with and without exact re-ranking.

Codes	Re-rank	Bytes/vector	Recall@10	ms/query
float32	–	512	1.000	0.71
SQ8	–	128	0.986	1.77
SQ8	4	128	1.000	1.85
PQ32	–	32	0.536	4.67
PQ32	4	32	0.905	4.54
PQ16	–	16	0.297	2.02
PQ16	4	16	0.628	1.91
PQ8	–	8	0.139	1.06
PQ8	4	8	0.349	1.07

Table 10: Compression vs. recall. Re-rank r = exact re-scoring of the top $r \cdot k$ candidates.

SQ8 cuts memory 4× at recall 0.986, and 1.000 with re-ranking. PQ32 (16× smaller) needs re-ranking to reach 0.905, and PQ16 and PQ8 lose most neighbours. These synthetic vectors are isotropic within each cluster, which is the worst case for product quantization: there is no low-dimensional structure for the sub-codebooks to capture. Real CLIP embeddings are far more compressible.

9.8 Scale

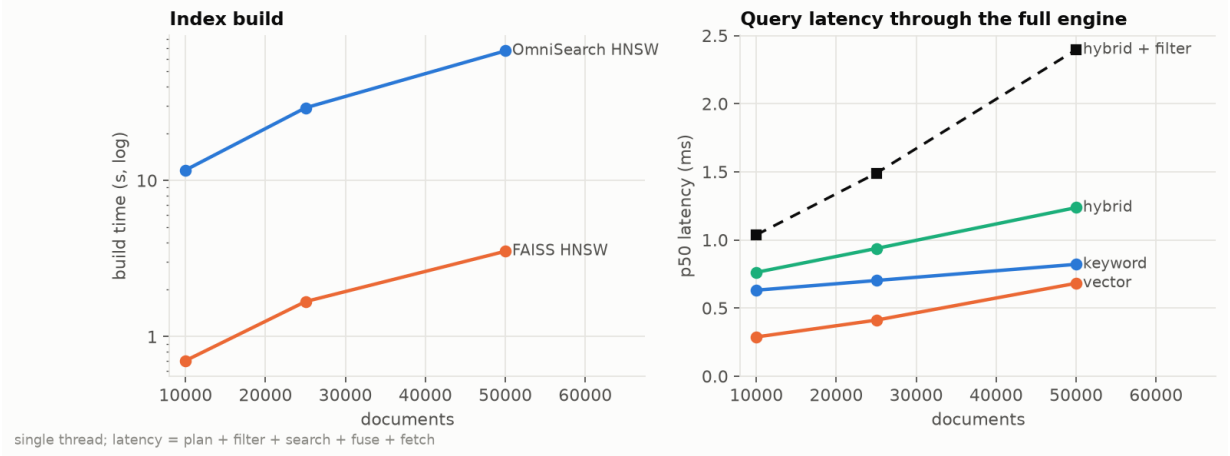


Figure 9: Build time, disk footprint and p50 query latency from 10k to 50k documents.

Docs	Ingest (s)	HNSW (s)	FAISS (s)	Disk (MB)	Keyword	Vector	Hybrid	+Filter
10,000	0.3	11.6	0.70	10.1	0.63	0.29	0.76	1.04
25,000	0.8	29.1	1.67	29.0	0.70	0.41	0.94	1.49
50,000	1.7	67.5	3.52	53.3	0.82	0.68	1.24	2.40

Table 11: Scaling. Build times in seconds; the last four columns are p50 query latency in ms.

Ingest and on-disk size grow linearly (53 MB at 50k documents). HNSW construction dominates build time: 67.5 s against 3.5 s for FAISS, with the same interpreted-graph overhead seen in Section 9.2. Query latency grows sub-linearly: a hybrid query takes 1.24 ms at 50k documents and 2.40 ms with a filter.

9.9 Retrieval quality on the demo collection

The systems experiments measure speed and exactness. This experiment measures *which retrieval path finds the right documents*. The collection is the 268-item demo collection built with the offline *classic* profile. Text is indexed from titles, descriptions and the OCR of every image and video keyframe (RapidOCR, PP-OCRv4). The vector side is `text_vec`, a 512-d hashed character-n-gram embedding of the same text. Relevance labels come from the data generator, never from the engine. Five query sets each probe a different path:

- **brand+category** (*"zenith sneaker"*): words present in titles;
- **colour+category** (*"red mug"*): a third of the titles omit the colour, which only the pixels show;
- **typos** (*"zenth sneker"*): the middle letter of every word of 4+ letters is deleted;
- **sign text (OCR)** (*"no parking"*): the text exists only inside the photo;
- **lecture (OCR)** (*"extendible hashing"*): the phrase appears only on a slide inside the video, never in its metadata.

Each query runs in five modes: keyword only (`MATCH`), vector only (`SIMILAR`), and hybrid with RRF, weighted and AUTO fusion.

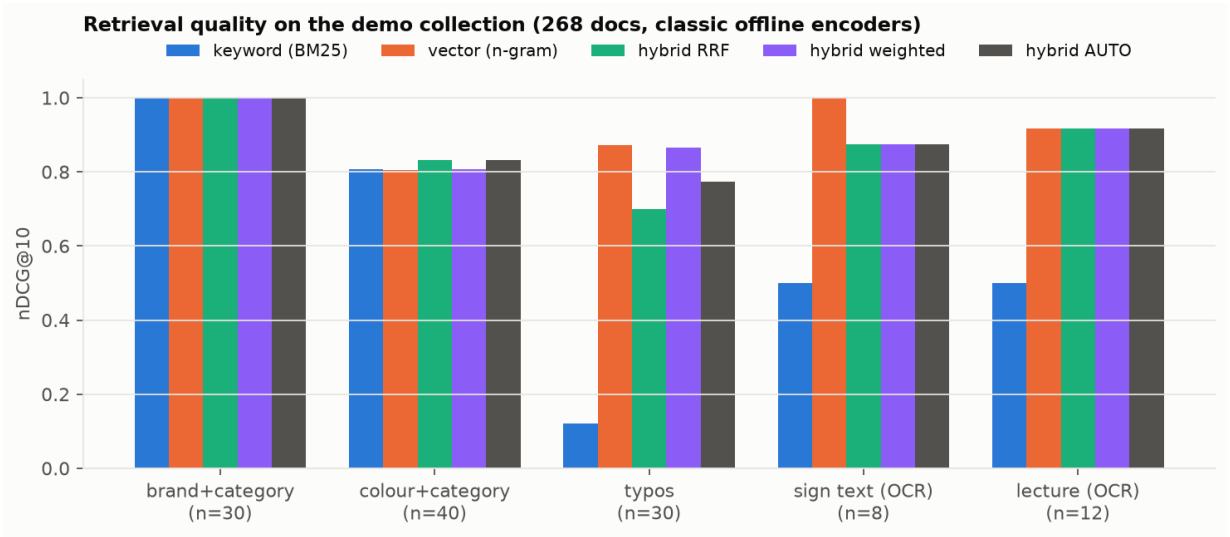


Figure 10: nDCG@10 per query set and retrieval mode on the demo collection.

Query set	Mode	R@1	R@10	MRR@10	nDCG@10	p50 ms
brand+category $n = 30$	Keyword (BM25)	1.000	1.000	1.000	**1.000**	1.45
	Vector	1.000	1.000	1.000	**1.000**	0.66
	Hybrid RRF	1.000	1.000	1.000	**1.000**	2.09
	Hybrid weighted	1.000	1.000	1.000	**1.000**	3.01
	Hybrid AUTO	1.000	1.000	1.000	**1.000**	1.99
colour+category $n = 40$	Keyword (BM25)	1.000	1.000	1.000	0.806	1.07
	Vector	1.000	1.000	1.000	0.806	0.53
	Hybrid RRF	1.000	1.000	1.000	0.831	1.51
	Hybrid weighted	1.000	1.000	1.000	0.807	2.66
	Hybrid AUTO	1.000	1.000	1.000	**0.831**	1.30
typos $n = 30$	Keyword (BM25)	0.100	0.400	0.162	0.121	0.50
	Vector	0.833	1.000	0.886	**0.872**	0.67
	Hybrid RRF	0.800	1.000	0.859	0.700	1.12
	Hybrid weighted	0.833	1.000	0.891	0.866	1.79
	Hybrid AUTO	0.800	1.000	0.868	0.773	1.67
sign text (OCR) $n = 8$	Keyword (BM25)	0.500	0.500	0.500	0.500	0.49
	Vector	1.000	1.000	1.000	**1.000**	0.89
	Hybrid RRF	0.875	0.875	0.875	0.875	1.02
	Hybrid weighted	0.875	0.875	0.875	0.875	2.02
	Hybrid AUTO	0.875	0.875	0.875	0.875	1.28
lecture (OCR) $n = 12$	Keyword (BM25)	0.500	0.500	0.500	0.500	0.28
	Vector	0.917	0.917	0.917	**0.917**	0.37
	Hybrid RRF	0.917	0.917	0.917	**0.917**	0.87
	Hybrid weighted	0.917	0.917	0.917	**0.917**	1.84
	Hybrid AUTO	0.917	0.917	0.917	**0.917**	1.00

Table 12: Retrieval quality on the demo collection. Bold: best nDCG@10 in each query set. Recall@k counts a query as answered if any relevant document is in the top k.

Clean queries are easy for every mode. Brand+category queries score nDCG@10 = 1.000 in every mode. Colour+category queries always find a relevant item at rank 1 (R@1 = 1.000), but nDCG@10 tops out at 0.83: the third of the products whose title omits the colour are unreachable from text, whatever the fusion. Text-to-pixel matching needs a cross-modal encoder (the CLIP profile).

Typos break the keyword path. Keyword search answers only 40% of the typo queries within the top 10 (nDCG@10 0.121), while character n-grams still overlap and the vector path reaches 0.872. Hybrid RRF falls to 0.700. The untouched category word (*"t-shirt"*) makes BM25 return confident but wrong-brand results, and rank-based fusion gives them the same weight as the correct vector ranking. For *"ace t-shirt"*, 4 of the 7 relevant items reach the top 10 with vectors alone, but only 1 after RRF. Weighted fusion with max-normalisation (0.866) is far more robust, because a low-scoring BM25 list contributes little.

OCR noise is the dominant real-world effect. The OCR model reads *NO PARKING* as the single token `NOPARKING` and *Node splits* as `Nodesplits`. The recognizer drops the spaces between words, and enabling RapidOCR's space-character option does not change that. Keyword search therefore answers only the sign texts whose words survived (*exit*, *stop*, *one way*, *sale*): nDCG@10 0.500 for signs and 0.500 for lecture slides. Worse, the query word *"no"* matches the product

model numbers `NO-1144`, so BM25 ranks Nova products first for *"no parking"*. The n-gram vectors are indifferent to the missing spaces and answer every sign query (1.000) and 11 of 12 lecture queries (0.917). The one miss, *"cost models"*, fails in every mode, because "models" also matches every product's "Model" code. Hybrid fusion recovers most of the vector path's quality (0.875 and 0.917) without being told which path to trust.

Top-10 neighbour shares the query's…	Precision@10	Chance
Same colour	0.718	0.125
Same category	0.865	0.200

Table 13: Query-by-example on the colour-layout image vectors, `SIMILAR(image_vec, DOC(id))`: the fraction of the 10 nearest images (excluding the query) that share the query image's colour or category, over 60 random product queries.

The image path works without any text. Query-by-example on the colour-layout vectors (Table 13) returns neighbours of the same colour 72% of the time (chance: 12.5%) and the same category 87% of the time (chance: 20%). The silhouette of a sneaker or a mug is captured by the spatial colour layout, even though no text is involved.

Takeaway. No single retrieval path wins everywhere. Lexical search is exact but brittle to typos and OCR noise; the vector path is robust but can be confused by shared sub-strings. Fusion is the safe default, and *how* it fuses matters: on noisy keyword lists, score-aware weighted fusion beats rank-only RRF.

9.10 Retrieval quality on public benchmarks

The engine includes loaders and an evaluation harness for four public cross-modal benchmarks (Table 14). For each benchmark, the harness ingests the corpus with captions, OCR and transcripts on the keyword side and CLIP / CLAP on the vector side. It then scores every caption query in the five modes of Section 9.9 with Recall@{1,5,10}, MRR@10 and nDCG@10. The captions used as queries are never ingested, which avoids answer leakage.

Benchmark	Modality	Test split	Notes
Flickr30k [12]	image	1,000 images × 5 captions	Karpathy split; the recommended headline benchmark
MS-COCO [13]	image	5,000 images × 5 captions	BLIP-base was fine-tuned on COCO captions, so keyword/hybrid scores are optimistic
MSR-VTT [14]	video	1,000 videos (1k-A)	keyframes every 1 s, multi-vector MaxSim
Clotho v2.1 [15]	audio	1,045 clips × 5 captions	Whisper + CLAP

Table 14: Supported public benchmarks.

These runs need the neural model profile (PyTorch, OpenCLIP, BLIP, Whisper; several GB of weights) and the datasets themselves, which require accepting their licences. Their results are not part of this report: all numbers above come from data the project generates itself. The commands to reproduce them are in Appendix A.

10 Discussion

The optimizer earns its keep. No fixed access path is safe. The plan that is best without a filter (HNSW) is 540× worse at 0.1% selectivity, and the plan that is best with a selective filter (exact scan) loses at full selectivity. A cost model with measured unit costs and decent selectivity estimates chose correctly at every point we tested. The estimates come first: the q-error results show that the statistics behind the choice matter as much as the cost formulas.

Work vs. time. WAND, NRA and HNSW do far less *work* than their exhaustive counterparts, but in an interpreted language they can still lose on *time* to a vectorised scan. The benchmarks therefore report work (postings scored, node expansions, sorted/random accesses) alongside latency, so the algorithmic comparison stays meaningful and the constant factors stay visible. In a compiled implementation, the crossovers in Figures 4 and 7 would move toward the index-based plans.

Exactness of fusion is a real issue. Fixed-depth fusion, the default in most hybrid search systems, silently drops up to 31% of the true top-50 when the lists disagree. TA gives exactness at a cost close to fixed-depth merging.

Hybrid search helps when the paths disagree. The quality study shows that the two retrieval paths fail in different ways. The keyword index needs exact tokens. The n-gram vector space tolerates typos and OCR noise but is blind to word boundaries and meaning. Fusion recovers most of the better path's quality without knowing in advance which path that is.

11 Limitations and future work

- **Speed.** The pure Python engine runs HNSW about 13× slower than FAISS per query. A C or Rust core for graph traversal, WAND and NRA is the obvious next step; the cost model would then simply re-calibrate.
- **Durability and concurrency.** The engine supports a single writer. The record store commits per insert, but index files are checkpointed by `save()` with no write-ahead log, and deletes are tombstones without compaction.
- **Filtered HNSW.** Under anti-correlated filters the index itself degrades; the optimizer avoids this by cost, but ACORN-style predicate-agnostic neighbour expansion [9] would fix the index.
- **Correlation-aware estimation.** Selectivities combine under independence; joint statistics or sampling would let the optimizer anticipate the correlation effect of Table 6.
- **Evaluation.** Systems results use synthetic data, and real-data retrieval quality requires the neural profile and licensed datasets (Section 9.10).

12 Conclusion

OmniSearch shows that hybrid multimodal search is best understood, and best engineered, as a database system. It reduces every modality to text and vectors, gives each a proper index with statistics, and puts a calibrated cost-based optimizer and exact top-k fusion on top. The experiments confirm the classic lessons in a new setting: access-path choice depends on selectivity and correlation, selectivity estimates must be good for that choice to be right, and approximate aggregation can be replaced by exact, instance-optimal algorithms at little cost.

References

[1] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[2] A. Z. Broder, D. Carmel, M. Herscovici, A. Soffer and J. Zien. Efficient query evaluation using a two-level retrieval process. In *CIKM*, 2003.

[3] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4), 2018.

[4] H. Jégou, M. Douze and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 33(1), 2011.

[5] R. Fagin, A. Lotem and M. Naor. Optimal aggregation algorithms for middleware. In *PODS*, 2001.

[6] G. V. Cormack, C. L. A. Clarke and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *SIGIR*, 2009.

[7] J. Wang et al. Milvus: A purpose-built vector data management system. In *SIGMOD*, 2021.

[8] C. Wei et al. AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data. *PVLDB*, 13(12), 2020.

[9] L. Patel, P. Kraft, C. Guestrin and M. Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. In *SIGMOD*, 2024.

[10] P. G. Selinger et al. Access path selection in a relational database management system. In *SIGMOD*, 1979.

[11] A. Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

[12] P. Young, A. Lai, M. Hodosh and J. Hockenmaier. From image descriptions to visual denotations. *TACL*, 2, 2014.

[13] T.-Y. Lin et al. Microsoft COCO: Common objects in context. In *ECCV*, 2014.

[14] J. Xu, T. Mei, T. Yao and Y. Rui. MSR-VTT: A large video description dataset for bridging video and language. In *CVPR*, 2016.

[15] K. Drossos, S. Lipping and T. Virtanen. Clotho: An audio captioning dataset. In *ICASSP*, 2020.

A Reproducing the results

On Windows, use `mingw32-make` in place of `make`.

```
make install                        # venv + dependencies
make test                           # 100 tests
make demo                           # 268-item demo collection (ffmpeg)
python -m benchmarks.run_all        # systems benchmarks (~20 min)
python -m benchmarks.demo_quality   # demo retrieval quality
python -m benchmarks.plots          # figures + results/SUMMARY.md
cd report && latexmk -pdf omnisearch_report.tex

# public benchmarks (neural models + datasets)
pip install -r requirements-models.txt
# place the Flickr30k images and Karpathy split under data/flickr30k
python -m benchmarks.retrieval_quality --dataset flickr30k --root data/flickr30k
```

B OmniQL quick reference

Clause	Meaning
`MATCH(field, 'text' [, weight=w])`	BM25 scoring clause on a text field
`SIMILAR(field, q [, weight, ef, nprobe])`	vector scoring; q is `'text'`, `IMAGE()`, `AUDIO()`, `DOC()`, `VECTOR[...]` or a weighted sum
`CONTAINS(field, 'words')`	hard keyword filter (all terms present)
`=, !=, <, <=, >, >=, IN, BETWEEN, IS [NOT] NULL`	attribute filters combined with `AND` / `OR` / `NOT` under three-valued logic
`FUSE RRF(k) | WEIGHTED(norm) | AUTO`	fusion of the scoring clauses
`WITH (option=value, ...)`	optimizer overrides: `strategy` (exact, index, postfilter), `topk` (full, ta, nra, exhaustive), `keyword_algo`, `filter_access`, `ef`, `nprobe`, `depth`